

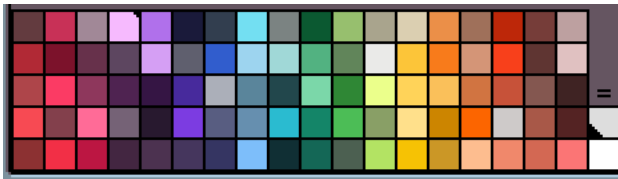
# GODOT DOKUMENTATION

-Karin Purgišová

## DESIGN

### FARBSHEMA

Eines der ersten Dinge, die ich getan habe, war die Erstellung einer Farbpalette, die für alle folgenden Designs verwendet werden sollte. Wir wollten, dass das Spiel eine helle und freundliche Atmosphäre hat, daher sind die meisten Farben relativ stark gesättigt.



### START-BUTTON

Das erste Element, das ich noch vor dem Farbschema erstellt habe, war ein Konzept für den Start-Button.

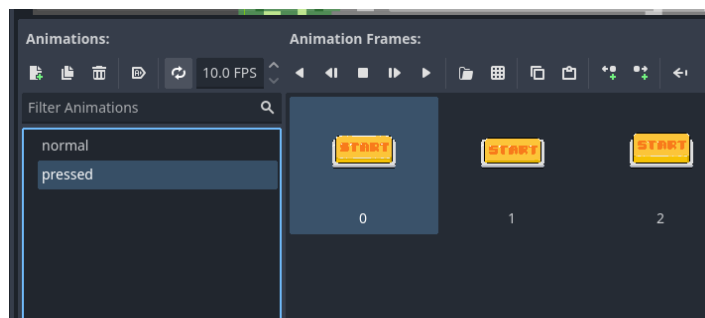
Erste Version:

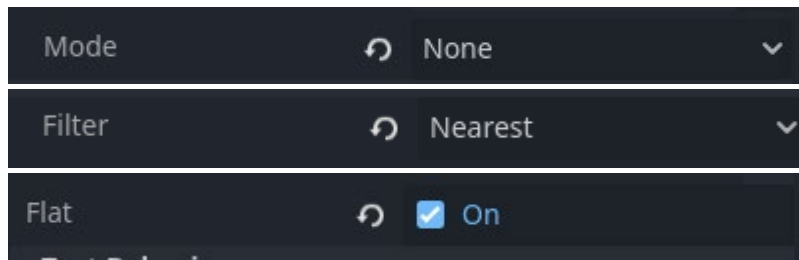


Mit der Erstellung des Farbschemas wurde auch der Start-Button verändert. Nicht nur farblich, sondern das gesamte Design wurde geändert, um die spätere Implementierung von Animationen lebendiger zu gestalten.



Der nächste Schritt war die Erstellung der Animation. Diese Animation wurde durch "AnimatedSprite2D" implementiert.





Ich musste die oben gezeigten Einstellungen innerhalb des Start-Button-Inspectors ändern, damit die voreingestellten Visuals das Aussehen der Animation nicht beeinträchtigen.

## HEALTH BARS

Als nächsten Schritt habe ich Gesundheitsbalken für den Spieler und die Gegner entworfen (sie unterscheiden sich nur in der Farbe). Diese health bars bestehen aus dem "Container" und der Füllung, um die Implementierung in den Code zu vereinfachen



## ENTEN-ZÄHLER SCHRIFTART

Ein weiteres wichtiges Asset, das wir benötigten, war der Zähler für die gesammelten Enten. Deshalb habe ich diese Schriftart erstellt, bei der ich sie gelb halten wollte, um die Verbindung zwischen den Enten und dem Sammler deutlicher zu machen.



## STARTBILDSCHIRM

Im Startbildschirm verwendete ich den zuvor erwähnten Start-Button, Hintergrund und Enten-Assets, die von meinen Kollegen erstellt wurden, sowie Label-Text und einen kleinen Glanz-Effekt. Zuerst erstellte ich den Startbildschirm ohne die Enten- und Glanz-Animation, jedoch wirkte diese Version etwas zu statisch, weshalb ich mich entschied, etwas Bewegung in Form dieser beiden Animationen hinzuzufügen.



## GAME OVER BILDSCHIRM

Ich erstellte eine Grafik für den Game Over Bildschirm sowie zwei Buttons.



Dies ist das vollständige Aussehen des Game Over Bildschirms im Spiel. Der rote Hintergrund ist teilweise transparent, sodass der Spieler beim Erscheinen noch teilweise die Karte sehen kann.



# CODE

## START-BUTTON

```
func _on_start_button_pressed():
>I  # plays pressed animation
>I  start_button_animation.play("pressed")
>I
>I  # plays button sound when pressed
>I  button_sound_player.play()
>I
>I  #delays hiding the button until animation finishes
>I  await start_button_animation.animation_finished
>I
>I  start_button.hide()
>I  $"Title Text".hide()
>I  background_pic.hide()
>I  duck.hide()
>I  shine.hide()
>I  start_game.emit()
```

(im Skript der "HUD"-Szene)

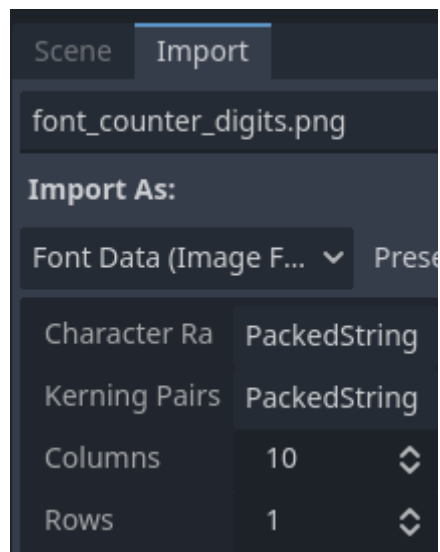
Im Code mussten wir die Button-Logik definieren (nicht der gesamte Code im Snippet wurde von mir implementiert) und wollten die Animation abspielen und erst danach die Aktionen ausführen. Das gewährleistet die "await"-Codezeile. Der Rest dient dazu, die Startbildschirm-Aspekte auszublenden.

## GESUNDHEITSBALKEN

*wurden nicht von mir im Code implementiert*

## ENTEN-ZÄHLER SCHRIFTART

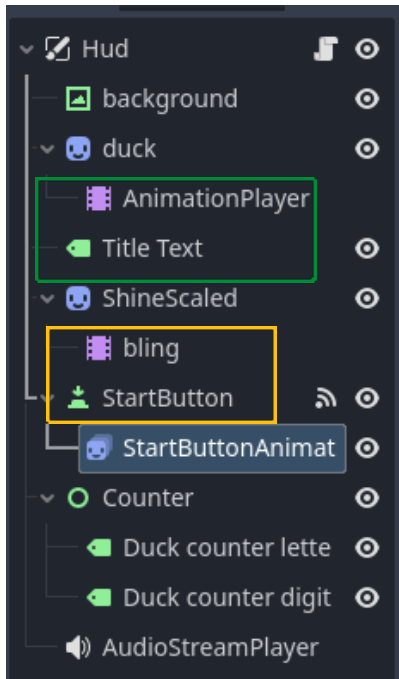
Ich hatte ein kleines Problem mit der Implementierung der Schriftart. Mein erstes Problem war das Format selbst, da es eine .png-Datei war. Dies war relativ einfach zu beheben, nachdem ich eine Lösung gefunden hatte – die Datei musste in Godot als Schriftdaten neu importiert werden.



Mein anderes Problem war, dass meine Schriftart nur aus Zahlen bestand und daher mein Text "Ducks" nicht angezeigt werden konnte. Deshalb entschied ich, dass die beste und einfachste Lösung darin bestand, den Text und die Zählung selbst in separate Labels zu trennen, was dieses Problem löste.

```
func duck_collected_func():
    >| ducks_collected += 1
    >| counter_letters.text = "Ducks: "
    >| counter_numbers.text = str(ducks_collected)
    >| #devided into two labels so that the use of the custom font for numbers only
    >| #would be possible
```

## STARTBILDSCHIRM



Im Code definierte ich, dass beide Animationen starten sollten, wenn der Bildschirm geladen wird.

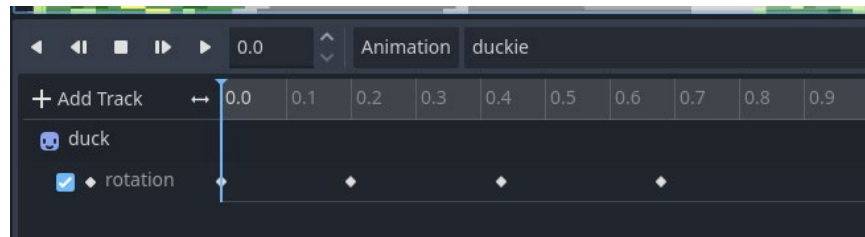
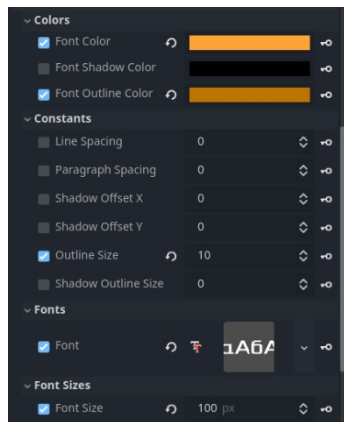
```
func _ready() -> void:  
    > GlobalSignals.duck_collected_signal.connect(duck_collected_func)  
    > duck_anim.play("duckie")  
    > shine_anim.play("bling")
```

Hier legte ich fest, dass alle Aspekte des Startbildschirms verschwinden sollten, nachdem der Start-Button gedrückt wurde.

```
func _on_start_button_pressed():
    >| # plays pressed animation
    >| start_button_animation.play("pressed")
    >|
    >| # plays button sound when pressed
    >| button_sound_player.play()
    >|
    >| #delays hiding the button until animation fin
    >| await start_button_animation.animation_finish

    >|
    >| start_button.hide()
    >| $"Title Text".hide()
    >| background_pic.hide()
    >| duck.hide()
    >| shine.hide()
    >| start_game.emit()
```

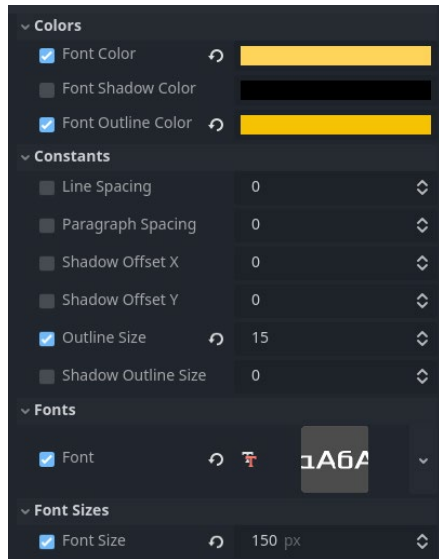
Beide Animationen sind auf Loop gesetzt. Ich verwendete die in Godot nativ verfügbaren Einstellungen, um das Aussehen des Titeltexs zu ändern.





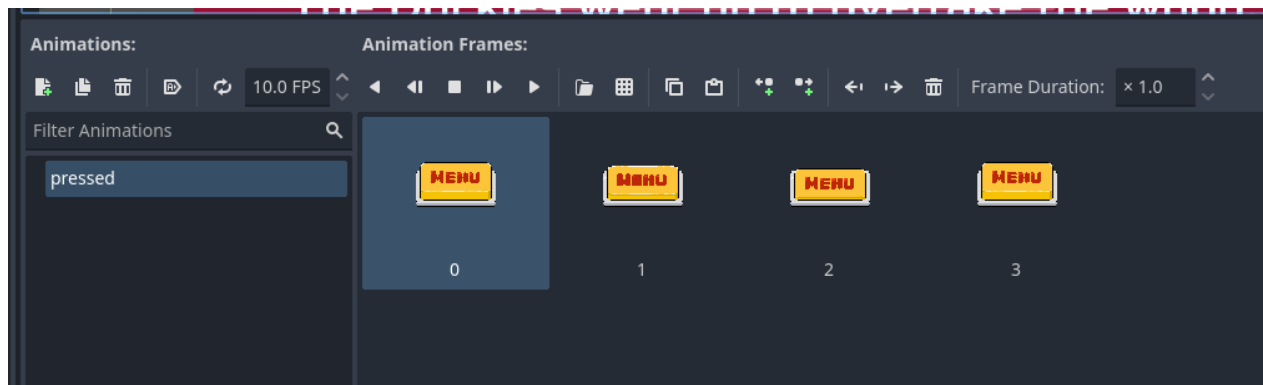
## GAME OVER BILDSCHIRM

Der Game Over Bildschirm war wahrscheinlich das schwierigste Element für mich zu implementieren, und ich habe mehr als eine Methode ausprobiert. Am Ende entschied ich mich dafür, die Szene zu einer Kindszene von Main zu machen und sie während der Spieldauer versteckt zu halten. Sie wird durch die Funktion `hide()` sichtbar gemacht, sobald der Spieler in eine Ente verwandelt wird (der Einfachheit halber als Spielertod bezeichnet).



Das Aussehen der Schriftart wurde erneut durch die nativen Features von Godot eingestellt.

Beide Animationen sind auf genau dieselbe Weise implementiert (mit `AnimatedSprite2D` und einem Sprite Sheet).



**Button-Funktionen:**

```

extends CanvasLayer

func _on_menu_pressed() -> void:
>| $Menu/AnimatedMenuButton.play("pressed")
>|
>| await get_tree().create_timer(0.5).timeout
>|
>| get_tree().change_scene_to_file("res://main.tscn")
>|
>|

func _on_quit_pressed() -> void:
>| $Quit/AnimatedQuitButton.play("pressed")
>|
>| await get_tree().create_timer(0.5).timeout
>| get_tree().quit()

```

- Wenn der Menu-Button gedrückt wird, wird die Main-Szene geladen – das bedeutet, der Startbildschirm wird angezeigt
- Wenn der Quit-Button gedrückt wird, wird das gesamte Spiel beendet

**Implementierung durch Signale:** Das Anzeigen des Game Over Bildschirms wird durch Signale implementiert.

```

func _ready():
>| #pass #replace with function if needed
>| $Hud.start_game.connect(new_game)
>| var trashcan = preload("res://Destroyable/destroyable_trashcan.tscn").instantiate()
>| trashcan.global_position = Vector2(2000,2400)
>| add_child(trashcan)
>| # Connect the player's death signal to show game over
>| $Player.health_death.connect(_on_player_died)

```

```

func _on_player_died():
>| $GameOver.show()

```

(in Main)

```

func _on_animation_player_animation_finished(anim_name: StringName) -> void:
>| health_death.emit()

```

(in Player)

Wenn das health\_death Signal ausgesendet wird, wird die Funktion \_on\_player\_died() aufgerufen, die den Game Over Bildschirm anzeigt.